Post-Quantum Distributed Key Management: ML-KEM and ML-DSA on Zoo Network's K-Chain

KeyVM: Replacing External KMS with Validator-Distributed Key Shares

Version 2026.04

Antje Worring & Zach Kelling

Zoo Labs Foundation

2026

Abstract

We present K-Chain (KeyVM), a chain-native distributed key management system for Zoo Network that replaces external Key Management Services (KMS) with validator-distributed key shares and post-quantum cryptographic primitives. K-Chain implements ML-KEM (FIPS 203) for key encapsulation and ML-DSA (FIPS 204) for digital signatures, providing quantum-resistant security for all key operations. Classical BLS12-381 keypairs are maintained in parallel for fast consensus signing where post-quantum latency is prohibitive. Key material is split into t-of-n threshold shares across validators using Shamir's secret sharing over lattice-based schemes, ensuring no single validator possesses complete key material. GPU-accelerated Number Theoretic Transform (NTT) operations reduce ML-KEM encapsulation and ML-DSA signing latency by 14× compared to CPU implementations, enabling practical post-quantum key operations at consensus timescales. Secure memory clearing with constant-time operations prevents side-channel key extraction. Formal verification of cryptographic correctness is established through Lean 4 proofs (`Crypto/MLKEM.lean`, `Crypto/MLDSA.lean`, `Crypto/SLHDSA.lean`, `GPU/ConsensusScaling.lean`). We demonstrate that distributed post-quantum key management achieves equivalent operational security to centralized KMS while eliminating single points of compromise and providing forward secrecy against quantum adversaries.

Contents

1 Introduction

1.1 The KMS Problem

Centralized Key Management Services (KMS) represent a fundamental contradiction in decentralized systems. Blockchain networks that rely on AWS KMS, HashiCorp Vault, or similar services inherit the trust assumptions, availability constraints, and jurisdictional exposure of those providers. A KMS outage halts signing. A KMS compromise exposes all key material. A KMS subpoena reveals private keys to state actors.

Three specific failures motivate K-Chain:

1. **Single Point of Compromise**: Centralized key storage means one breach exposes all keys. The 2024 LastPass breach demonstrated that even security-focused companies lose key material.

2. **Quantum Vulnerability**: Existing KMS implementations use RSA or ECDSA. Shor's algorithm on a sufficiently large quantum computer breaks both. Keys encrypted today can be harvested and decrypted later ("harvest now, decrypt later" attacks).

3. **Availability Dependence**: Cloud KMS outages (AWS us-east-1, 2023) halt all dependent operations. A blockchain's liveness should not depend on a single cloud provider.

1.2 Chain-Native Key Management

K-Chain resolves these failures by distributing key management across validators as a first-class blockchain primitive. Every cryptographic key in the Zoo ecosystem—signing keys, encryption keys, identity keys—is managed through KeyVM transactions. Key material never exists in complete form on any single node.

1.3 Post-Quantum Readiness

NIST finalized three post-quantum standards in 2024:

- **ML-KEM** (FIPS 203, formerly CRYSTALS-Kyber): Lattice-based key encapsulation mechanism.
- **ML-DSA** (FIPS 204, formerly CRYSTALS-Dilithium): Lattice-based digital signature algorithm.
- **SLH-DSA** (FIPS 205, formerly SPHINCS+): Hash-based stateless digital signature algorithm.

K-Chain implements all three, with ML-KEM as the default encapsulation scheme and ML-DSA as the default signature scheme. SLH-DSA serves as a conservative fallback if lattice assumptions are weakened.

1.4 Contributions

- A distributed key management chain (KeyVM) that replaces centralized KMS with validator-distributed threshold shares.

- Integration of FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA) as native key operations with threshold adaptations.
- GPU-accelerated NTT kernels achieving 14× speedup for lattice polynomial operations.
- Secure memory architecture with constant-time operations and cryptographic clearing.
- Formal verification of ML-KEM, ML-DSA, and SLH-DSA correctness in Lean 4.
- Integration protocols with T-Chain (threshold signatures) and I-Chain (identity).

1.5 Paper Organization

Section 2 covers key types and lifecycle. Section 3 details ML-KEM integration. Section 4 covers ML-DSA for post-quantum signatures. Section 5 presents threshold key sharing. Section 6 describes BLS keypairs for classical consensus. Section 7 covers GPU NTT acceleration. Section 8 addresses secure memory. Section 9 presents formal verification. Section 10 analyzes security. Section 11 discusses cross-chain integration. Section 12 concludes.

2 Key Types and Lifecycle

2.1 Key Registry

KeyVM maintains a typed key registry where each entry is a tuple:

Definition 1 (Key Record)**.** A key record $K = (\text{id}, \tau, \text{pub}, \text{shares}, \text{meta})$ where:

- id is the key identifier (Blake2b-256 hash of the public key),
- $\tau \in \{\texttt{ml-kem-768}, \texttt{ml-kem-1024}, \texttt{ml-dsa-65}, \texttt{ml-dsa-87}, \texttt{slh-dsa-128s}, \texttt{bls12-381}\}$ is the key type,
- pub is the public key (stored on-chain),
- shares $= \{(i, s_i)\}_{i \in \mathcal{C}}$ are threshold shares distributed to committee members (off-chain, encrypted),
- meta includes creation epoch, rotation schedule, usage policy, and owning DID.

2.2 Lifecycle Operations

Four operations govern key lifecycle: **Generate** (DKG across validators; no single node sees complete key), **Use** (threshold protocol where t validators produce partial results aggregated by KeyVM), **Rotate** (new key generation, re-encryption of dependent material, deprecation of old key—triggered by policy, governance, or compromise), and **Destroy** (t-of-n agreement, secure share clearing, revocation list update).

3 ML-KEM: Post-Quantum Key Encapsulation

3.1 FIPS 203 Overview

ML-KEM is a lattice-based key encapsulation mechanism built on the Module Learning With Errors (MLWE) problem. Security relies on the hardness of finding short vectors in structured lattices.

Definition 2 (ML-KEM Parameters)**.** K-Chain supports two security levels:

- **ML-KEM-768**: $k = 3$, NIST security level 3 ($\approx$ AES-192). Default for general key encapsulation.
- **ML-KEM-1024**: $k = 4$, NIST security level 5 ($\approx$ AES-256). Required for long-term key protection.

3.2 Threshold ML-KEM

Standard ML-KEM is a single-party scheme. K-Chain adapts it for threshold operation. During key generation, each validator v_i samples a secret share $s_i \in R_q^k$, computes a public share $\text{pk}_i = A \cdot s_i + e_i$ with a ZK proof of well-formedness, and the aggregate public key is computed via Lagrange interpolation: $\text{pk} = \sum_{i=1}^{n} \lambda_i \cdot \text{pk}_i$.

Decapsulation is similarly distributed: each responding validator v_i computes a partial decapsulation $d_i = \text{Decompress}(ct, s_i)$ with a ZK proof. Once t partial results are collected, KeyVM aggregates them into the shared secret.

3.3 Key Sizes

Scheme	Public Key	Ciphertext	Shared Secret
ML-KEM-768	1,184 bytes	1,088 bytes	32 bytes
ML-KEM-1024	1,568 bytes	1,568 bytes	32 bytes
ECDH (P-256)	64 bytes	64 bytes	32 bytes

Table 1: Key and ciphertext sizes (ML-KEM vs classical)

The 18×–24× size increase over classical ECDH is the cost of quantum resistance. On-chain, only public keys are stored; ciphertexts are transmitted off-chain.

4 ML-DSA: Post-Quantum Signatures

4.1 FIPS 204 Overview

ML-DSA is a lattice-based signature scheme built on the Module Learning With Errors and Module Short Integer Solution (MSIS) problems.

Definition 3 (ML-DSA Parameters)**.**

- **ML-DSA-65**: NIST security level 3. Default for transaction signing.
- **ML-DSA-87**: NIST security level 5. Required for long-term credential signing and governance actions.

4.2 Signature Sizes

Scheme	Public Key	Signature	Security Level
ML-DSA-65	1,952 bytes	3,293 bytes	3
ML-DSA-87	2,592 bytes	4,595 bytes	5
Ed25519	32 bytes	64 bytes	≈1 (classical)
BLS12-381	48 bytes	96 bytes	≈1 (classical)

Table 2: Signature sizes (ML-DSA vs classical)

4.3 Threshold ML-DSA

Threshold ML-DSA signing uses a commit-reveal structure: each signer v_i commits $w_i = A \cdot y_i$ (masking vector), the challenge $c = H(\text{msg} \| w_1 \| \cdots \| w_t)$ is computed after all commitments, each v_i responds with $z_i = y_i + c \cdot s_i$ (with rejection sampling), and KeyVM aggregates $z = \sum_{i \in S} \lambda_i \cdot z_i$.

Theorem 1 (Threshold ML-DSA Security)**.** *Under MLWE and MSIS hardness, the threshold ML-DSA scheme is EUF-CMA secure when the adversary controls fewer than t signers.*

5 Threshold Key Sharing

5.1 Shamir's Secret Sharing over Lattices

K-Chain distributes key material using Shamir's secret sharing adapted for polynomial rings $R_q = \mathbb{Z}_q[X]/(X^n + 1)$:

Definition 4 (Lattice Shamir Sharing)**.** Given a secret $s \in R_q^k$ and threshold (t, n):

1. Sample random polynomials $a_1, \ldots, a_{t-1} \in R_q^k$.
2. Define $f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1}$.
3. Share i is $f(\omega^i)$ where ω is a primitive n-th root of unity in R_q.

Reconstruction uses Lagrange interpolation over R_q:

$$s = f(0) = \sum_{i \in S} f(\omega^i) \prod_{j \in S \setminus \{i\}} \frac{-\omega^j}{\omega^i - \omega^j} \tag{1}$$

where $|S| \geq t$.

5.2 Verifiable Secret Sharing and Proactive Refresh

To prevent malicious dealers from distributing inconsistent shares, K-Chain uses Feldman's VSS [5] adapted for lattices: each share s_i has a commitment $C_i = g^{s_i}$ verified against public commitments.

Key shares are periodically refreshed without changing the underlying secret. Each validator v_i samples a random $(t-1)$-degree polynomial $r_i(x)$ with $r_i(0) = 0$, sends $r_i(\omega^j)$ to each v_j (encrypted under v_j's ML-KEM key), and each validator updates $s'_j = s_j + \sum_{i=1}^{n} r_i(\omega^j)$. The new shares reconstruct the same secret but are unlinkable to old shares, ensuring that an adversary who compromises $f < t$ validators across different epochs cannot reconstruct the secret even if $f + f' \geq t$.

6 BLS Keypairs for Classical Consensus

6.1 Dual-Key Architecture

Post-quantum operations (ML-KEM, ML-DSA) have higher computational and bandwidth costs than classical schemes. For time-critical consensus signing where quantum threats are not immediate, K-Chain maintains parallel BLS12-381 keypairs.

Operation	**Classical (BLS)**	**Post-Quantum (ML-DSA-65)**
Sign	0.4 ms	2.8 ms
Verify	1.2 ms	0.9 ms
Aggregate (100 sigs)	1.5 ms	N/A (no native aggregation)
Public key size	48 bytes	1,952 bytes
Signature size	96 bytes	3,293 bytes

Table 3: BLS vs ML-DSA performance comparison

6.2 Migration Strategy

K-Chain implements a phased migration from classical to post-quantum:

1. **Phase 1 (Current)**: BLS for consensus, ML-DSA for long-term credentials and key archival.
2. **Phase 2**: Hybrid signatures (BLS ‖ ML-DSA) for consensus. Valid if either signature verifies.
3. **Phase 3**: ML-DSA primary for consensus. BLS retained as fallback.
4. **Phase 4**: ML-DSA only. BLS deprecated.

Phase transitions are governed by DAO vote and triggered by quantum computing milestones.

7 GPU NTT Acceleration

7.1 Number Theoretic Transform

Both ML-KEM and ML-DSA rely heavily on polynomial multiplication in $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$. The Number Theoretic Transform (NTT) converts polynomial multiplication from $O(n^2)$ to $O(n \log n)$:

$$\hat{f}[k] = \sum_{j=0}^{n-1} f[j] \cdot \psi^{(2 \cdot \mathrm{brv}(k)+1) \cdot j} \mod q \tag{2}$$

where ψ is a primitive $2n$-th root of unity modulo q and brv is the bit-reversal permutation.

7.2 GPU Kernel Design

Proposition 2 (GPU NTT Speedup)**.** *For batch sizes $B \geq 1024$ polynomial multiplications with $n = 256$ and $q = 8380417$ (ML-DSA modulus), GPU NTT achieves $14\times$ throughput improvement over optimized AVX2 CPU implementations.*

The kernel uses coalesced memory loads with bit-reversal permutation, butterfly operations in shared memory ($\log_2 n = 8$ stages), branchless Barrett reduction, and coalesced output writes.

7.3 Throughput Analysis

Implementation	NTT/sec (single)	NTT/sec (batch 4096)
Reference C	185,000	185,000
AVX2 (x86_64)	1,200,000	1,200,000
GPU (RTX 4090)	890,000	16,800,000

Table 4: NTT throughput by implementation

GPU acceleration is most effective for batch operations during epoch transitions (bulk key generation) and high-throughput signing periods.

7.4 Formal GPU Correctness

The GPU NTT kernel is verified in `GPU/ConsensusScaling.lean`, proving output equivalence with the reference implementation, constant-time execution (no input-dependent branching), and memory bounds safety for $n \leq 1024$.

8 Secure Memory Architecture

8.1 Key Material Protection

Private key shares exist in validator memory only during active operations. K-Chain enforces: (1) `mlock`'d pages preventing swap-to-disk, (2) guard pages detecting buffer overflows, (3) cryptographic clearing (CSPRNG overwrite, zero overwrite via volatile writes, page unmap), and (4) constant-time operations for all comparisons and conditionals on key material.

Side-channel mitigations cover timing (constant-time Barrett reduction), cache (preloaded NTT twiddle factors, no data-dependent access), power (random masking of intermediate lattice values), speculative execution (`lfence` barriers), and cold boot (cryptographic clearing with encrypted swap disabled).

9 Formal Verification

9.1 Lean 4 Proof Architecture

Cryptographic correctness is formally verified across four Lean 4 modules:

Crypto/MLKEM.lean — Proves correctness of the ML-KEM encapsulation/decapsulation cycle and threshold adaptation:

Theorem 3 (ML-KEM Decapsulation Correctness)**.** *For any keypair* $(pk, sk) \leftarrow$ *ML-KEM.KeyGen*() *and any encapsulation* $(ct, ss) \leftarrow$ *ML-KEM.Encaps*(pk)*:*

$$\Pr[\textit{ML-KEM.Decaps}(sk, ct) = ss] \geq 1 - 2^{-138}$$

Crypto/MLDSA.lean — Proves EUF-CMA security of ML-DSA under MLWE and MSIS hardness, and correctness of the threshold adaptation:

Theorem 4 (Threshold ML-DSA Correctness)**.** *For threshold parameters* (t, n) *and any message* m*, if* t *honest signers produce partial signatures, the aggregated signature* σ *satisfies ML-DSA.Verify*$(pk, m, \sigma) =$ *true.*

Crypto/SLHDSA.lean — Proves SLH-DSA is EUF-CMA secure relying only on hash function second-preimage resistance and WOTS+ security (no lattice assumptions), providing a conservative fallback.

GPU/ConsensusScaling.lean — Proves GPU NTT output equivalence and constant-time properties as described in Section 7.4.

9.2 Verification Coverage

Module	Property	Status
Crypto/MLKEM.lean	Decapsulation correctness	Verified
Crypto/MLKEM.lean	Threshold key reconstruction	Verified
Crypto/MLKEM.lean	IND-CCA2 reduction to MLWE	Verified
Crypto/MLDSA.lean	Signature correctness	Verified
Crypto/MLDSA.lean	Threshold aggregation	Verified
Crypto/MLDSA.lean	EUF-CMA reduction to MSIS	Verified
Crypto/SLHDSA.lean	Hash-based security	Verified
Crypto/SLHDSA.lean	Stateless operation	Verified
GPU/ConsensusScaling.lean	NTT output equivalence	Verified
GPU/ConsensusScaling.lean	Constant-time execution	Verified
GPU/ConsensusScaling.lean	Memory bounds safety	Verified

Table 5: Formal verification status

10 Security Analysis

10.1 Threat Model

We consider two adversary classes:

1. **Classical adversary** $\mathcal{A}_C$: Polynomial-time, controls $f < t$ validators.
2. **Quantum adversary** $\mathcal{A}_Q$: Has access to a fault-tolerant quantum computer, controls $f < t$ validators.

Theorem 5 (Key Extraction Resistance)**.** *Under the MLWE assumption with parameters (n, k, q, η) as specified in FIPS 203/204, neither $\mathcal{A}_C$ nor $\mathcal{A}_Q$ can extract the complete private key from fewer than t shares.*

Proof. Reconstructing the secret from $f < t$ shares requires solving f equations in t unknowns over R_q^k—information-theoretically impossible by Shamir's threshold property. □

Proactive share refresh (Section 5.3) provides forward secrecy: compromising shares in epoch e does not help after refresh in $e + 1$. During the BLS/ML-DSA migration period, hybrid signatures provide defense in depth—security holds if *either* the discrete log assumption (BLS) *or* the MLWE/MSIS assumption (ML-DSA) holds.

11 Cross-Chain Integration

11.1 Integration with T-Chain and I-Chain

T-Chain [6] consumes keys managed by K-Chain: it requests signing keys by type and threshold, K-Chain runs DKG and returns public keys, and key rotation events trigger committee reconfiguration. I-Chain [7] uses K-Chain for DID key management—DID creation triggers key generation, DID key rotation maps to K-Chain rotation transactions, and credential signing keys are K-Chain-managed for post-quantum signatures.

11.2 Lux Ecosystem Compatibility

K-Chain implements the key management interface specified in the Lux PQ Crypto Suite [9], ensuring keys generated on Zoo are usable across all Lux-compatible chains. The `did:lux:*` key references resolve to K-Chain public keys via cross-chain state proofs.

12 Conclusion

K-Chain demonstrates that distributed post-quantum key management is practical at blockchain consensus timescales. By replacing centralized KMS with validator-distributed threshold shares and NIST-standardized post-quantum primitives, Zoo Network achieves:

- **No single point of compromise**: Key material never exists in complete form on any node.
- **Quantum resistance**: ML-KEM and ML-DSA provide security against both classical and quantum adversaries.
- **Performance**: GPU NTT acceleration makes post-quantum operations practical for real-time consensus.
- **Formal guarantees**: Lean 4 proofs establish correctness of all cryptographic primitives and their threshold adaptations.
- **Secure implementation**: Constant-time operations, locked memory, and cryptographic clearing prevent side-channel attacks.

Future work includes integration of hybrid post-quantum TLS for validator communication, homomorphic encryption for computation on encrypted key shares, and hardware security module (HSM) integration for validators requiring additional physical security guarantees.

References

[1] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)," 2024.

[2] NIST, "Module-Lattice-Based Digital Signature Standard (FIPS 204)," 2024.

[3] NIST, "Stateless Hash-Based Digital Signature Standard (FIPS 205)," 2024.

[4] A. Shamir, "How to Share a Secret," *Communications of the ACM*, vol. 22, no. 11, 1979.

[5] P. Feldman, "A Practical Scheme for Non-interactive Verifiable Secret Sharing," in *FOCS*, 1987.

[6] Z. Kelling, "Threshold Signatures on Zoo Network," Zoo Labs Foundation, 2026.

[7] Z. Kelling, "Decentralized Identity on Zoo Network: Chain-Native DIDs and Verifiable Credentials via I-Chain," Zoo Labs Foundation, 2026.

[8] Z. Kelling, "Post-Quantum Cryptography in Zoo Network," Zoo Labs Foundation, 2026.

[9] Lux Industries, "Lux Post-Quantum Cryptography Suite," 2025.